

ADVANCED PROGRAMMING
ASSIGNMENT 6
NAME- BHITALI BORUAH
ROLLNO-CSB24045

Assignment –6

```
import java.io.PrintStream;
import java.util.AbstractMap;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Set;
import java.util.stream.Collectors;

public class StudentPerformanceAnalyzer {
    public StudentPerformanceAnalyzer() {
    }

    public static List<Student> getTopNStudents(List<Student> students, int n)
    {
        return (List) (students != null && !students.isEmpty() && n > 0 ?
        (List) students.stream().sorted(Comparator.comparingDouble(Student::getAverageScore).reversed()).limit((long) n).collect(Collectors.toList()) : new
        ArrayList());
    }

    public static Map<String, Double> getAverageScorePerCourse(List<Student>
    students) {
        if (students != null && !students.isEmpty()) {
            Map<String, List<Integer>> scoresPerCourse =
            (Map) students.stream().flatMap((student) -> {
                return student.getCourses().stream().map((course) -> {
                    return new AbstractMap.SimpleEntry(course,
                    (Integer) student.getScores().getOrDefault(course, 0));
                });
            }).collect(Collectors.groupingBy(Map.Entry::getKey,
            Collectors.mapping(Map.Entry::getValue, Collectors.toList())));
            return
            (Map) scoresPerCourse.entrySet().stream().collect(Collectors.toMap(Map.Entry::g
            etKey, (entry) -> {
                return
                ((List) entry.getValue()).stream().mapToInt(Integer::intValue).average().orElse
                (0.0);
            }));
        } else {
            return new HashMap();
        }
    }
}
```

```

    public static Set<String> getAllUniqueCourses(List<Student> students) {
        return (Set) (students != null && !students.isEmpty() ?
(Set)students.stream().flatMap((student) -> {
            return student.getCourses().stream();
        }).collect(Collectors.toCollection(HashSet::new)) : new HashSet());
    }

    public static Map<String, Double>
getAverageScorePerCourseTraditional(List<Student> students) {
    Map<String, List<Integer>> scoresPerCourse = new HashMap();
    Iterator var2 = students.iterator();

    while(var2.hasNext()) {
        Student student = (Student)var2.next();
        Iterator var4 = student.getCourses().iterator();

        while(var4.hasNext()) {
            String course = (String)var4.next();
            int score = (Integer)student.getScores().getOrDefault(course,
0);

            ((List)scoresPerCourse.computeIfAbsent(course, (k) -> {
                return new ArrayList();
            })).add(score);
        }
    }

    Map<String, Double> averages = new HashMap();
    Iterator var8 = scoresPerCourse.entrySet().iterator();

    while(var8.hasNext()) {
        Map.Entry<String, List<Integer>> entry = (Map.Entry)var8.next();
        double avg =
((List)entry.getValue()).stream().mapToInt(Integer::intValue).average().orElse
(0.0);
        averages.put((String)entry.getKey(), avg);
    }

    return averages;
}

    public static void main(String[] args) {
        List<Student> students = createSampleStudents();
        System.out.println("Top 3 Students:");
        List<Student> topStudents = getTopNStudents(students, 3);
        PrintStream var10001 = System.out;
        Objects.requireNonNull(var10001);
        topStudents.forEach(var10001::println);
        System.out.println("\nAverage Scores per Course:");
    }
}

```

```

        Map<String, Double> courseAverages =
getAverageScorePerCourse(students);
        courseAverages.forEach((course, avg) -> {
            System.out.printf("%s: %.2f%n", course, avg);
        });
        System.out.println("\nAll Unique Courses:");
        Set<String> uniqueCourses = getAllUniqueCourses(students);
        var10001 = System.out;
        Objects.requireNonNull(var10001);
        uniqueCourses.forEach(var10001::println);
    }

    private static List<Student> createSampleStudents() {
        List<Student> students = new ArrayList();
        List<String> courses1 = Arrays.asList("Math", "Physics", "Chemistry");
        Map<String, Integer> scores1 = new HashMap();
        scores1.put("Math", 85);
        scores1.put("Physics", 90);
        scores1.put("Chemistry", 78);
        students.add(new Student(1, "Alice", courses1, scores1));
        List<String> courses2 = Arrays.asList("Math", "Physics", "Biology");
        Map<String, Integer> scores2 = new HashMap();
        scores2.put("Math", 92);
        scores2.put("Physics", 88);
        scores2.put("Biology", 95);
        students.add(new Student(2, "Bob", courses2, scores2));
        List<String> courses3 = Arrays.asList("Math", "Chemistry", "Biology");
        Map<String, Integer> scores3 = new HashMap();
        scores3.put("Math", 78);
        scores3.put("Chemistry", 82);
        scores3.put("Biology", 79);
        students.add(new Student(3, "Charlie", courses3, scores3));
        List<String> courses4 = Arrays.asList("Physics", "Chemistry",
"Biology");
        Map<String, Integer> scores4 = new HashMap();
        scores4.put("Physics", 85);
        scores4.put("Chemistry", 88);
        scores4.put("Biology", 84);
        students.add(new Student(4, "Diana", courses4, scores4));
        return students;
    }
}

```

Output

Top 3 Students:

Student{id=2, name='Bob', avgScore=91.67}

Student{id=4, name='Diana', avgScore=85.67}

Student{id=1, name='Alice', avgScore=84.33}

Average Scores per Course:

Chemistry: 82.67

Biology: 86.00

Math: 85.00

Physics: 87.67

All Unique Courses:

Chemistry

Biology

Math

Physics

Process finished with exit code 0